

5강

데이터 정제 & 시각화

□ 결측치 처리

dropna / fillna



이상치 탐지

IQR / Z-score



matplotlib

line / bar / scatter



seaborn

heatmap / boxplot

오늘의 학습 목표

01 □ 데이터 정제

- 1 결측치(NaN)를 찾고 제거·대체할 수 있다
- 2 이상치를 IQR 기법으로 탐지할 수 있다
- 3 dropna / fillna / clip 을 활용한다

02 📊 기초 시각화

- 1 matplotlib으로 선·막대·산점도를 그린다
- 2 seaborn으로 분포·상관관계를 시각화한다
- 3 그래프에 제목·축·범례를 추가한다

03 🔗 정제 → 시각화 파이프라인

- 1 정제된 데이터로 바로 시각화한다
- 2 결측치·이상치를 시각적으로 확인한다
- 3 분석 보고용 차트를 저장한다

데이터 분석 파이프라인



□ 왜 정제(Cleaning)가 중요한가?

📊 결측치가 있으면 평균·합계 계산 결과가 왜곡됩니다.

🔑 이상치가 포함되면 머신러닝 모델 성능이 크게 떨어집니다.

📊 정제하지 않은 데이터로 그린 차트는 잘못된 패턴을 보여줍니다.

결측치(Missing Value)란?

① 결측치

데이터에 값이 없거나 비어있는 상태 — pandas에서는 NaN(Not a Number)으로 표시

NaN이 포함된 DataFrame 예시

이름	나이	점수	지역
김민준	25	92	서울
이서연	NaN	85	경기
박지훈	31	NaN	NaN
최수아	28	77	부산

📋 결측치 발생 원인

- 설문 응답 누락 (응답 거부, 실수)
- 센서 오류 · 전송 중 손실
- 데이터 병합 시 키 불일치
- 해당 없음 (N/A) 처리 미흡

📊 결측치 유형 (3가지)

MCAR

완전 무작위 결측

어떤 패턴도 없이 랜덤하게 발생

MAR

무작위 결측

다른 변수와 관련있지만 해당 변수와 무관

MNAR

비무작위 결측

해당 값 자체의 특성으로 발생 (가장 문제)

결측치 확인하기

① 결측치

먼저 어디에 얼마나 있는지 파악하는 것이 첫 번째 단계

```
import pandas as pd

df = pd.read_csv("sample_data.csv")

# ① 결측치 여부 확인 (True/False 표)
print( df.isnull() )

# ② 열별 결측치 개수
print( df.isnull().sum() )

# ③ 결측치 비율 (%)
missing_rate = df.isnull().sum() / len(df) * 100
print( missing_rate.round(2) )

# ④ 결측치가 있는 행만 보기
print( df[ df.isnull().any(axis=1) ] )

# ⑤ 전체 결측치 총수
print("총 결측치:", df.isnull().sum().sum())
```

📄 출력 예시 (isnull().sum())

열	결측 수	비율
이름	0	0.0%
나이	0	0.0%
점수	5	3.3%
지역	0	0.0%

💡 시각적으로 결측치 확인하기

```
import seaborn as sns
import matplotlib.pyplot as plt

# 결측치 히트맵
sns.heatmap(df.isnull(),
            yticklabels=False,
            cbar=False, cmap='viridis')
plt.title("결측치 위치 (노란색=NaN)")
plt.show()
```

결측치 처리 전략 — 제거 vs 대체

① 결측치

상황에 따라 전략이 달라집니다 — 하나의 정답은 없어요

제거 (Deletion)

- `df.dropna()` → 결측치 있는 행 전체 제거
- `df.dropna(subset=["점수"])` → 특정 열 기준
- `df.dropna(thresh=3)` → 3개 이상 있어야 유지

- ✓ 결측치가 5% 미만일 때
- ✓ MCAR (완전 무작위) 결측일 때
- ⊖ 데이터가 부족하거나 패턴이 있을 때

대체 (Imputation)

- `df.fillna(0)` → 0으로 채우기
- `df.fillna(df["점수"].mean())` → 평균으로
- `df.fillna(df["점수"].median())` → 중앙값
- `df.fillna(method="ffill")` → 앞값으로 채우기

- ✓ 데이터 수가 적을 때
- ✓ 수치형 열에 평균·중앙값 사용
- ✓ 시계열은 `ffill/bfill` 권장

처리 전략 선택 기준

결측 비율 5% 이하?

→ `dropna()` 제거 권장

5~30% 결측?

→ `fillna(평균·중앙값)` 대체

30% 이상?

→ 열 삭제 또는 모델 활용 검토

dropna() & fillna() — 핵심 코드

① 결측치

실제로 가장 많이 쓰는 패턴을 정리했습니다

dropna() — 제거

```
# 결측치 있는 행 모두 제거
df_clean = df.dropna()

# 특정 열에 결측치 있는 행만 제거
df_clean = df.dropna(subset=["점수"])

# 최소 n개 값이 있어야 행 유지
df_clean = df.dropna(thresh=3)

# 열 제거 (axis=1)
df_clean = df.dropna(axis=1)

# 원본도 바꾸기 (inplace)
df.dropna(inplace=True)

# 제거 전후 비교
print(f"전: {df.shape} → 후: {df_clean.shape}")
```

fillna() — 대체

```
# 고정값으로 채우기
df["점수"].fillna(0, inplace=True)

# 평균으로 채우기 (수치형 권장)
mean_score = df["점수"].mean()
df["점수"].fillna(mean_score, inplace=True)

# 중앙값으로 채우기 (이상치 있을 때)
median = df["점수"].median()
df["점수"].fillna(median, inplace=True)

# 최빈값으로 채우기 (범주형)
mode = df["지역"].mode()[0]
df["지역"].fillna(mode, inplace=True)

# 앞/뒤 값으로 채우기 (시계열)
df.fillna(method="ffill", inplace=True)
```

💡 팁: fillna 후 확인 → `df.isnull().sum()` 으로 0이 되었는지 반드시 검증하세요!

이상치(Outlier)란?

② 이상치

다른 데이터와 현저히 다른 값 — 분석 결과를 크게 왜곡할 수 있습니다

📌 이상치 예시

점수 데이터:

85	91	78
88	250 ⚠️	82
76	-5 ⚠️	90

250점, -5점 → 불가능한 값이지만 데이터에 존재!

이상치가 미치는 영향

- 평균: 85 → (이상치 포함) 93.4 (크게 왜곡)
- 표준편차: 5.2 → 71.8 (분포 완전히 달라짐)
- 시각화: 정상 데이터가 작은 점으로 뭉침

📋 이상치 발생 원인

- 입력 오류 (타이핑 실수: 85 → 850)
- 측정 장비 오류 (센서 오작동)
- 데이터 전처리 오류 (단위 혼용)
- 실제로 특이한 케이스 (정상 이상치)

🔍 이상치 탐지 방법 (3가지)

IQR 방법

$Q1 - 1.5 \times IQR \sim Q3 + 1.5 \times IQR$ 범위 벗어나면 이상치
→ 분포 모양에 무관하게 적용 가능

Z-score

평균에서 표준편차 3배 이상 벗어난 값
→ 정규분포를 따를 때 효과적

시각화

Boxplot · Scatter plot으로 육안 확인
→ 가장 직관적, 먼저 그려보는 것 권장

IQR 방법으로 이상치 탐지 & 처리

② 이상치

가장 많이 쓰이는 이상치 탐지 방법 — 사분위수 기반

IQR (사분위 범위) 개념

$$\text{IQR} = Q3 - Q1$$

- 하한 경계 = $Q1 - 1.5 \times \text{IQR}$
- 상한 경계 = $Q3 + 1.5 \times \text{IQR}$
- 이 범위 밖의 값 = 이상치 ⚠

```
# IQR 계산
Q1 = df["점수"].quantile(0.25)
Q3 = df["점수"].quantile(0.75)
IQR = Q3 - Q1

# 경계값 설정
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

# 이상치 확인
outliers = df[(df["점수"] < lower) |
               (df["점수"] > upper)]
print(f"이상치 {len(outliers)}개:", outliers)
```

🔗 이상치 처리 3가지 방법

방법 1 — 제거 (Drop)

```
df_clean = df[(df["점수"] >= lower) &
               (df["점수"] <= upper)]
```

방법 2 — 클리핑 (Clipping) ★ 권장

```
# clip: 범위 밖 → 경계값으로 대체
df["점수"] = df["점수"].clip(lower, upper)
```

방법 3 — 대체 (Replace)

```
# 이상치 위치를 NaN으로 바꾼 뒤 fillna
df.loc[df["점수"] > upper, "점수"] = None
df["점수"].fillna(df["점수"].median(),
                  inplace=True)
```

💡 클리핑은 데이터를 없애지 않고 범위 내로 제한하기 때문에 정보 손실이 가장 적어 실무에서 자주 씁니다.

시각화 라이브러리 — matplotlib vs seaborn

③ 시각화

두 라이브러리는 함께 사용하는 경우가 많습니다

matplotlib

특징

파이썬 표준 시각화 라이브러리
가장 세밀한 제어 가능

주요 차트

plt.plot() → 선 그래프
plt.bar() → 막대 그래프
plt.scatter() → 산점도
plt.hist() → 히스토그램

```
import matplotlib.pyplot as plt
plt.figure(figsize=(8, 5))
plt.plot(x, y, color='blue')
plt.title("제목")
plt.xlabel("X축")
plt.ylabel("Y축")
plt.legend()
plt.show()
```

seaborn

특징

matplotlib 기반의 고수준 라이브러리
통계 시각화에 특화, 코드가 짧음

주요 차트

sns.histplot() → 히스토그램+분포
sns.boxplot() → 박스플롯
sns.heatmap() → 히트맵
sns.scatterplot() → 산점도

```
import seaborn as sns
import matplotlib.pyplot as plt
sns.set_theme(style="whitegrid")
sns.boxplot(data=df, x="지역",
            y="점수", palette="Set2")
plt.title("지역별 점수 분포")
plt.show()
```

matplotlib — 핵심 차트 4종

③ 시각화

matplotlib

모든 차트에서 plt.show() 또는 plt.savefig()로 출력합니다

선 그래프 (plt.plot)

```
x = [1, 2, 3, 4, 5]
y = [10, 25, 18, 30, 22]
plt.figure(figsize=(6, 4))
plt.plot(x, y,
         color='steelblue',
         linewidth=2,
         marker='o')
plt.title("월별 매출 추이")
plt.xlabel("월")
plt.show()
```

막대 그래프 (plt.bar)

```
labels = ['A', 'B', 'C', 'D']
values = [30, 55, 42, 70]
plt.figure(figsize=(6, 4))
plt.bar(labels, values,
        color=['#3E86F5']*4,
        edgecolor='white')
plt.title("카테고리별 판매량")
plt.xlabel("카테고리")
plt.ylabel("판매량")
plt.show()
```

산점도 (plt.scatter)

```
plt.figure(figsize=(6, 4))
plt.scatter(
    df["나이"], df["점수"],
    color='orange',
    alpha=0.6,
    s=50
)
plt.title("나이 vs 점수")
plt.xlabel("나이")
plt.show()
```

히스토그램 (plt.hist)

```
plt.figure(figsize=(6, 4))
plt.hist(
    df["점수"],
    bins=20,
    color='purple',
    edgecolor='white',
    alpha=0.7
)
plt.title("점수 분포")
plt.show()
```

seaborn — 핵심 차트 4종

③ 시각화

seaborn

`sns.set_theme()`으로 스타일을 먼저 설정하면 더 예쁘게 나와요

히스토그램+분포 (histplot)

```
sns.histplot(
    data=df,
    x="점수",
    kde=True,      # 분포 곡선 추가
    bins=20,
    color='purple'
)
plt.title("점수 분포 + 곡선")
plt.show()
```

박스플롯 (boxplot)

```
sns.boxplot(
    data=df,
    x="지역",
    y="점수",
    palette="Set2"
)
plt.title("지역별 점수 분포")
plt.xticks(rotation=30)
plt.show()
```

히트맵 (heatmap)

```
# 상관관계 히트맵
corr = df.select_dtypes("number").corr()

sns.heatmap(
    corr,
    annot=True,    # 수치 표시
    fmt=".2f",
    cmap="coolwarm"
)
plt.title("변수 간 상관관계")
plt.show()
```

산점도+회귀선 (regplot)

```
sns.regplot(
    data=df,
    x="나이",
    y="점수",
    scatter_kws={"alpha":0.5},
    line_kws={"color":"red"}
)
plt.title("나이 vs 점수 + 추세선")
plt.show()
```

여러 차트 한 번에 — 서브플롯 & 저장

③ 시각화

분석 보고서에서 가장 많이 쓰는 패턴입니다

서브플롯 (여러 그래프를 한 화면에)

```
import matplotlib.pyplot as plt

# 2행 2열 서브플롯 생성
fig, axes = plt.subplots(
    nrows=2, ncols=2,
    figsize=(12, 8)
)

# 각 subplot에 그래프 그리기
axes[0, 0].hist(df["점수"], bins=20)
axes[0, 0].set_title("점수 분포")

axes[0, 1].scatter(df["나이"], df["점수"])
axes[0, 1].set_title("나이 vs 점수")

axes[1, 0].bar(df["지역"].value_counts().index,
               df["지역"].value_counts().values)
axes[1, 0].set_title("지역별 인원")

axes[1, 1].boxplot(df["점수"].dropna())
axes[1, 1].set_title("점수 박스플롯")

plt.tight_layout() # 겹침 자동 조정
plt.show()
```

차트 저장하기

```
# show() 대신 savefig()로 파일 저장
plt.figure(figsize=(8, 5))
plt.plot(x, y)
plt.title("제목")

# PNG 저장 (고해상도)
plt.savefig(
    "chart.png",
    dpi=150, # 해상도
    bbox_inches="tight" # 여백 자동 제거
)

# PDF 저장 (벡터 이미지)
plt.savefig("chart.pdf", bbox_inches="tight")
plt.close() # 메모리 해제
```

seaborn 스타일 설정

```
sns.set_theme(style="whitegrid") # 흰 배경 + 격자
sns.set_theme(style="darkgrid") # 어두운 배경
sns.set_palette("Set2") # 색상 팔레트
```

정제 → 시각화 — 전체 파이프라인

① 정제

→ ③ 시각화

데이터 읽기부터 차트 저장까지 한 번에 이어보기

① 읽기

read_csv



② 탐색

info/shape



③ 결측치

dropna/fillna



④ 이상치

IQR/clip



⑤ 시각화

plot/show



⑥ 저장

savefig

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_theme(style="whitegrid")
# ① 읽기
df = pd.read_csv("sample_data.csv")
print(f"크기: {df.shape}   결측: {df.isnull().sum().sum()}") # ② 탐색
# ③ 결측치 처리
df["점수"].fillna(df["점수"].median(), inplace=True)
# ④ 이상치 처리 (IQR 클리핑)
Q1, Q3 = df["점수"].quantile([0.25, 0.75])
IQR = Q3 - Q1
df["점수"] = df["점수"].clip(Q1-1.5*IQR, Q3+1.5*IQR)
# ⑤ 시각화 (서브플롯 3개)
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
df["점수"].hist(ax=axes[0], bins=20, color='steelblue')
axes[0].set_title("점수 분포")
sns.boxplot(data=df, x="지역", y="점수",
            ax=axes[1], palette="Set2")
axes[1].set_title("지역별 점수")
axes[1].tick_params(axis='x', rotation=30)
corr = df.select_dtypes("number").corr()
sns.heatmap(corr, annot=True, ax=axes[2], cmap="coolwarm")
axes[2].set_title("상관관계")
plt.tight_layout()
plt.savefig("result.png", dpi=150, bbox_inches="tight") # ⑥ 저장
plt.show()
```

어떤 차트를 써야 할까? — 차트 선택 가이드

③ 시각화

목적에 맞는 차트를 고르는 것이 시각화의 핵심

분포 확인

히스토그램
`hist / histplot`

언제? 점수, 나이, 금액 등
한 변수의 분포 모양 볼 때

비교

막대 그래프
`bar / barplot`

언제? 카테고리별 평균, 합계
비교할 때

추세

선 그래프
`plot / lineplot`

언제? 시간에 따른 변화
트렌드 볼 때

관계

산점도 + 회귀선
`scatter / regplot`

언제? 두 수치 변수 간
관계·상관 볼 때

이상치

박스플롯
`boxplot`

언제? 이상치 시각 확인
그룹별 분포 비교

상관관계

히트맵
`heatmap`

언제? 여러 변수 간
상관계수 한눈에

EDA — 탐색적 데이터 분석 실전 패턴

③ 시각화

EDA

분석 시작 전 데이터를 이해하는 표준 절차

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv("sample_data.csv")

# — STEP 1: 기본 탐색 —————
print(df.shape)
df.info()
print(df.describe())

# — STEP 2: 결측치 확인 —————
print(df.isnull().sum())

# — STEP 3: 분포 확인 —————
df.select_dtypes("number").hist(
    bins=20, figsize=(12, 6))
plt.tight_layout()
plt.show()

# — STEP 4: 범주형 변수 확인 —————
for col in df.select_dtypes("object").columns:
    print(df[col].value_counts())

# — STEP 5: 상관관계 확인 —————
corr = df.select_dtypes("number").corr()
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.show()
```

📋 EDA 체크리스트

STEP 1

기본 구조 파악

- shape — 행/열 수
- info() — 타입·결측치
- describe() — 통계 요약

STEP 2

결측치 처리

- isnull().sum() 확인
- 비율 계산 (5% 기준)
- dropna / fillna 처리

STEP 3

분포 확인

- 수치형: 히스토그램
- 범주형: value_counts()
- 이상치: 박스플롯

STEP 4

관계 파악

- 상관관계 히트맵
- 산점도 (주요 변수 쌍)
- 그룹별 비교 boxplot

5강 정리 — 오늘 배운 것

□ 결측치 처리

- `isnull().sum()` → 결측치 개수 확인
- `dropna()` → 결측치 행 제거 (5% 이하)
- `fillna(mean/median)` → 값으로 대체
- 처리 후 `isnull().sum()` 으로 반드시 검증!

📐 이상치 탐지

- $IQR = Q3 - Q1$ / $경계 = Q1 \pm 1.5 \times IQR$
- `clip(lower, upper)` → 클리핑 (데이터 유지)
- boxplot으로 시각적 확인 먼저!
- z-score: $\text{평균} \pm 3\sigma$ 벗어나면 이상치

📊 시각화

- matplotlib: `plt.plot/bar/scatter/hist`
- seaborn: `histplot/boxplot/heatmap/regplot`
- `subplots()` 로 여러 차트 한 화면에
- `savefig()` 로 PNG/PDF 저장 가능

💡 정제 없는 시각화는 거짓말입니다. 항상 정제 → 탐색 → 시각화 순서를!

📅 다음 시간 (6강) — 5강 실습: 결측치 처리 + 이상치 탐지 + 시각화 코드 직접 완성!